بسمه تعالی

درس طراحی سیستم‌های نهفته مبتنی بر FPGA

آزمایش ۱: طراحی کنترلر ارتباطی I2C و راه‌اندازی دوربین OV7670

دانشکدگان فنی دانشگاه تهران

دانشکده مهندسی برق و کامپیوتر

دکتر بیژن علیزاده

دستیاران آموزشی:

پارسا مندوئی	pmandooie@gmail.com	@Parsa_mandooie
حسین غرابادیان	hoseingharabadian@gmail.com	@hossein_GBDN
کیمیا فیروزبخش	Kimiyafiroozbakhsh@gmail.com	@Kimia_fbs
علی حصیری		

نیم‌سال اول ۱۴۰۶-۱۴۰۵

مدت آزمایش: چهار جلسه اهداف آزمایش:

- ✓ آشنایی با محیط توسعه Xilinx Vivado.
- ✓ آشنایی با پروتکل ارتباط سریال I2C و طراحی هسته سخت‌افزاری آن.
- ✓ بررسی ساختار کاری و پروتکل SCCB در سنسورهای تصویر دوربین OV7670.
- ✓ پیاده‌سازی روی بورد سخت‌افزاری و بررسی سیگنال‌ها با ماژول ILA (بخش امتیازی).

مقدمه

در این آزمایش با طراحی سخت‌افزاری در نرم‌افزار Vivado آشنا می‌شوید و یک کنترلر برای پروتکل ارتباط سریال I2C پیاده‌سازی می‌کنید. این آزمایش در دو بخش انجام می‌شود: در **بخش اول**، هسته I2C Master (i2c_master_core) را به زبان Verilog توصیف کرده و با طراحی یک Testbench و قرار دادن آن در کنار یک ماژول Slave ارائه‌شده، آن را شبیه‌سازی می‌کنید (نوشتن و خواندن Registerهای Slave). سپس یک ماشین حالت طراحی می‌کنید که با استفاده از این هسته، پروتکل SCCB را پیاده‌سازی کرده و شناسه‌های دوربین را می‌خوانید و آن را با ماژول ارائه‌شده‌ای که رفتار دوربین OV7670 را شبیه‌سازی می‌کند، آزمایش می‌کنید. در **بخش دوم**، وارد پیاده‌سازی سخت‌افزاری می‌شوید؛ ماژول‌ها را در محیط Block Design نرم‌افزار Vivado در کنار هم قرار داده و سیم‌کشی می‌کنید، فایل Bitstream را روی بورد Program کرده و با اتصال واقعی دوربین OV7670، خواندن شناسه‌های سنسور (PID و VER) را روی بورد تست می‌کنید (همچنین می‌توانید سیگنال‌ها را با ماژول ILA نیز به صورت زنده روی بورد بررسی نمایید).

۱ مفاهیم پایه و معماری ارتباط سریال

۱-۱ مقدمه: ارتباطات سریال در سیستم‌های نهفته و پردازش تصویر

در سیستم‌های نهفته مبتنی بر FPGA، ارتباط با ادوات جانبی نظیر سنسورها و کنترلرها از طریق رابط‌های سریال صورت می‌پذیرد. پروتکل‌های ارتباطی سریال نظیر I2C، SPI و UART نقش مهمی در ارسال داده‌های کنترلی و تنظیمات دارند. سنسورهای تصویر و دوربین‌های دیجیتال (مانند ماژول OmniVision OV7670) برای پیکربندی اولیه، تنظیمات نوردهی و فرمت تصویر، نیازمند یک درگاه کنترلی سریال هستند. در این آزمایش، کنترلر سخت‌افزاری این ارتباط سریال را بر پایه پروتکل استاندارد I2C پیاده‌سازی می‌کنیم تا مقدمه‌ای برای راه‌اندازی دوربین و دریافت فریم‌های تصویر در آزمایش‌های بعدی باشد.

۱-۲ مبانی و ساختار فیزیکی پروتکل I2C

پروتکل I2C (Inter-Integrated Circuit) یک Bus ارتباط سریال سنکرون و دو سیمه است که از دو خط تشکیل شده است:

- SCL (Serial Clock Line): خط سیگنال کلاک برای سنکرون‌سازی انتقال داده‌ها که معمولا توسط Master تولید می‌شود.
- SDA (Serial Data Line): خط دوطرفه انتقال آدرس و داده که میان فرستنده و گیرنده به اشتراک گذاشته می‌شود.

۱-۲-۱ ساختار Open-Drain و مقاومت‌های Pull-up

خطوط SCL و SDA هر دو از ساختار کلکتور/درین باز (Open-Drain) استفاده می‌کنند. تفاوت این ساختار با پایه‌های معمولی دیجیتال در این است که قطعات متصل به سیم، تنها می‌توانند خط را به صفر منطقی (زمین) وصل کنند و نمی‌توانند مستقیماً ولتاژ یک منطقی (۳.۳ ولت) روی خط قرار دهند. بنابراین برای برقراری سطح یک، از یک مقاومت بالاکش (Pull-up) متصل به خط تغذیه استفاده می‌شود؛ سازوکار این ساختار بدین ترتیب است:

- برای ارسال بیت «۰»، تراشه ترانزیستور خروجی خود را وصل کرده و سیم را به زمین وصل میکند.
- برای ارسال بیت «۱»، تراشه سیم را رها می‌کند (اتصال به زمین را قطع می‌کند) تا مقاومت بالاکش ولتاژ سیم را بالا کشیده و به یک منطقی برساند.

مزیت بسیار مهم این ساختار این است که اگر دو قطعه همزمان به خط مشترک متصل باشند، اتصال کوتاهی بین تغذیه و زمین ایجاد نمی‌شود و قطعات دچار سوختگی یا آسیب ولتاژی نمی‌گردند.

در بوردهای آموزشی آزمایشگاه، روی کانکتورهای اتصال دوربین مقاومت بالاکش فیزیکی وجود ندارد. در نتیجه اگر مقاومت بالاکش داخلیِ خودِ تراشه FPGA را در فایل Constraints فعال نکنید، خطوط در حالت شناور (معلق) باقی مانده و دوربین هیچ داده‌ای را دریافت یا ارسال نخواهد کرد. نحوه فعال‌سازی این مقاومت‌ها در بخش سوم تشریح گردیده است.

۱-۲-۲ شرایط شروع، پایان و انتقال بیت‌ها

۱. **اعتبار داده** (Data Validity): در هنگام انتقال داده، مقدار خط SDA در تمام طول نیم‌پالس بالای SCL باید پایدار بماند. تغییر وضعیت خط داده تنها در زمان پایین بودن خط کلاک SCL مجاز است.

۲. **شرط شروع** (START): تغییر وضعیت خط SDA از سطح یک به صفر در حالی که خط SCL در سطح یک منطقی قرار دارد.

۳. **شرط پایان** (STOP): تغییر وضعیت خط SDA از سطح صفر به یک در حالی که خط SCL در سطح یک منطقی است.

۴. **بیت تایید** (ACK/NACK): پس از ارسال هر ۸ بیت داده، در پالس نهم کلاک، فرستنده خط SDA را رها می‌کند. اگر گیرنده خط را به سطح صفر بکشد، به نشانه تایید دریافت بایت (ACK) است؛ در غیر این صورت، بالا ماندن خط علامت عدم تایید (NACK) خواهد بود.

۱-۳ Buffer دوطرفه IOBUF در تراشه‌های FPGA

پایه‌های فیزیکی متصل به هدرهای بیرونی تراشه FPGA به صورت دوطرفه (inout) تعریف می‌شوند. برای پیاده‌سازی خطوط درین باز در محیط Vivado، از ماژول استاندارد IOBUF استفاده می‌گردد. این ماژول دارای چهار Port اصلی است:

- IO: پایه فیزیکی متصل به Pin خارجی تراشه (inout).
- O: سیگنال خروجی به سمت منطق داخلی، نشان‌دهنده مقدار منطقی خوانده‌شده از Pin.
- I: داده خروجی به سمت Pin خارجی (در خطوط درین باز، همواره روی مقدار صفر ثابت بسته می‌شود).
- T: سیگنال کنترل Buffer سه‌حالته (Tri-state): اگر صفر شود، Buffer خروجی فعال شده و مقدار ورودی I را روی خط درایو می‌کند (چون I همواره صفر است، خط به زمین وصل می‌شود)؛ اگر یک شود، سیم رها می‌شود (حالت امپدانس بالا یا High-Z) تا مقاومت بالاکش خط را به یک منطقی برساند.

IOBUF یک Primitive از کتابخانه Xilinx است و به طور مستقیم به عنوان یک Block در Block Design در دسترس نیست. به همین دلیل در این آزمایش ماژول Verilog ساده‌ای با نام my_IOBUF طراحی می‌شود که تنها یک نمونه (Instance) از IOBUF را در خود دارد. Portهای این ماژول مطابق جدول ۱-۱ مستقیماً به Portهای IOBUF وصل می‌شوند.

در شبیه‌سازی نیازی به IOBUF نیست: خطوط Bus در Testbench به صورت tri1 (سیم با مقاومت بالاکش) تعریف می‌شوند و هر قطعه رفتار IOBUF را با یک انتساب ساده مدل می‌کند، برای مثال:

```
assign sda = sda_t ? 1'bz : sda_o;    assign sda_i = sda;
```

جدول ۱-۱: اتصال Portهای ماژول my_IOBUF به Portهای IOBUF

Port ماژول my_IOBUF	جهت	Port متناظر IOBUF	نقش
io_in	ورودی	I	داده به سمت Pin (در این آزمایش همواره صفر)
io_t	ورودی	T	کنترل سه‌حالته: ۰ درایو خط با مقدار I، ۱ رها کردن خط (High-Z)
io_out	خروجی	O	مقدار خوانده‌شده از Pin
io_io	دوطرفه	IO	پایه فیزیکی خارجی (inout)

۱-۴ پروتکل SCCB در دوربین OV7670 و تفاوت آن با I2C

دوربین OmniVision OV7670 جهت اعمال تنظیمات کنترلی از پروتکل اختصاصی SCCB (Serial Camera Control Bus) بهره می‌برد. لایه فیزیکی این پروتکل (دو خط درین‌باز، شرایط START و STOP، زمان‌بندی بیت‌ها) و قالب بسته‌های آن تقریباً همانند I2C است و به همین دلیل یک Master استاندارد I2C می‌تواند با دوربین ارتباط برقرار کند؛ با این حال چند تفاوت مهم وجود دارد:

- **بیت نهم:** در SCCB هر بایت (Phase) از ۸ بیت داده و یک بیت نهم تشکیل می‌شود. برخلاف I2C، این بیت نهم در مشخصات SCCB «بی‌اهمیت» (Don’t care) تعریف شده است. با این حال دوربین OV7670 در عمل در بیت نهم خط SDA را به صفر می‌کشد، بنابراین Master شما در این بیت یک ACK مشاهده می‌کند.

- در پروتکل SCCB، شرط شروع مجدد (Repeated START) در عملیات خواندن پشتیبانی نمی‌شود. بنابراین بازخوانی یک Register در قالب دو تراکنش کاملاً مجزا با صدور شرایط شروع و پایان مستقل صورت می‌پذیرد:

  ۱. **تراکنش اول (تعیین آدرس** Register**):** ارسال شرط START، آدرس نوشتن سنسور (0x42)، آدرس Register هدف، و صدور شرط STOP.

  ۲. **تراکنش دوم (خواندن بایت داده):** پس از STOP تراکنش اول، ارسال یک شرط START جدید، آدرس خواندن سنسور (0x43)، دریافت بایت داده، ارسال بیت NACK توسط Master، و صدور شرط STOP.

- **نوشتن:** در هر تراکنش نوشتن تنها یک بایت داده در یک Register نوشته می‌شود (ارسال آدرس 0x42، آدرس Register و یک بایت داده).

- **پیش‌نیازهای دسترسی به دوربین:** دوربین تنها زمانی به SCCB پاسخ می‌دهد که کلاک XCLK به آن اعمال شده باشد؛ بنابراین پیش از هر تراکنش باید XCLK برقرار باشد. همچنین پایه #RESET دوربین باید در سطح یک و پایه PWDN در سطح صفر باشد.

- Register**های شناسه دوربین:** سنسور OV7670 دارای دو Register فقط‌خواندنی (Read-Only) جهت احراز هویت ماژول است:

- PID Register (Product ID) در آدرس 0x0A با مقدار ثابت 0x76.
- VER Register (Version) در آدرس 0x0B با مقدار ثابت 0x73.

جزئیات پروتکل SCCB در جلسه پیش از آزمایش ارائه شده است. در صورت نیاز به جزئیات بیشتر، به سند OmniVision Serial Camera Control Bus (SCCB) Functional Specification و دیتاشیت دوربین OV7670 مراجعه کنید.

۱-۵ ماژول ILA (Integrated Logic Analyzer) – بخش امتیازی

ماژول تحلیل‌گر منطقی داخلی (ILA) ابزاری قدرتمند درون FPGA است که نمونه‌های سیگنال‌های دیجیتال را در حین اجرای واقعی سخت‌افزار در Blockهای حافظه BRAM ذخیره کرده و از طریق رابط JTAG به محیط نرم‌افزار Vivado Hardware Manager ارسال می‌نماید. در این آزمایش، پیاده‌سازی و تست با ماژول ILA به عنوان یک بخش **امتیازی** (Bonus) در نظر گرفته شده است؛ تایید اصلی صحت کارکرد مدار و پایان موفقیت‌آمیز خواندن اطلاعات، با روشن شدن یک نشانگر LED بر روی بورد صورت می‌گیرد.

۲ پیاده‌سازی و شبیه‌سازی هسته I2C (بخش اول)

برای راه‌اندازی دوربین OV7670 و پیاده‌سازی کاربردهای پردازش تصویر در آزمایش‌های آینده، ابتدا لازم است تنظیمات داخلی سنسور دوربین (شامل ابعاد تصویر، نرخ فریم، فرمت رنگی و میزان نوردهی) را پیکربندی کنیم. تمام این تنظیمات از طریق نوشتن و خواندن در Registerهای داخلی سنسور و به کمک پروتکل سریال SCCB (که از نظر لایه فیزیکی و قالب بسته‌ها تقریباً با پروتکل استاندارد I2C سازگار است؛ بخش ۱-۴) انجام می‌شود. با توجه به این‌که در بافت تراشه FPGA به طور پیش‌فرض کنترلر سخت‌افزاری آماده‌ای برای این پروتکل وجود ندارد، در گام نخست باید خودمان یک هسته سخت‌افزاری اختصاصی به زبان Verilog طراحی کنیم تا بتواند به عنوان Master، وظیفه تولید کلاک SCL، ایجاد شرایط شروع و پایان و ارسال و دریافت بایت‌های داده روی خطوط Bus را بر عهده بگیرد. همچنین باید یک ماشین حالت برای کنترل ماژول I2C جهت اعمال پروتکل SCCB و خواندن رجیسترهای دوربین طراحی شود.

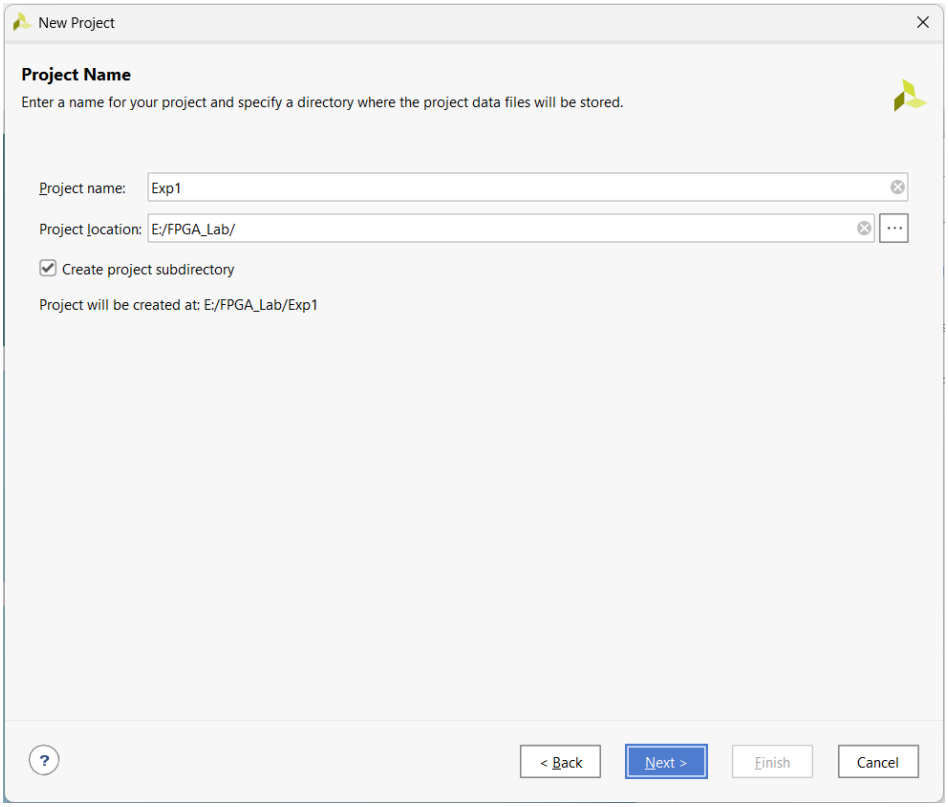
پیش از پیاده‌سازی این مدار روی بورد واقعی، برای اطمینان کامل از درستی عملکرد منطقی و رعایت دقیق زمان‌بندی پروتکل، ابتدا یک پروژه در محیط Vivado ایجاد کرده و هسته و ماشین حالت طراحی‌شده را در Testbench با اتصال به ماژول‌های Slave ارائه‌شده شبیه‌سازی و اعتبارسنجی می‌کنیم. در تمام این آزمایش، فرکانس کلاک SCL باید 100 kHz (Standard-mode) باشد.

۱-۲ ایجاد پروژه در Vivado و انتخاب قطعه

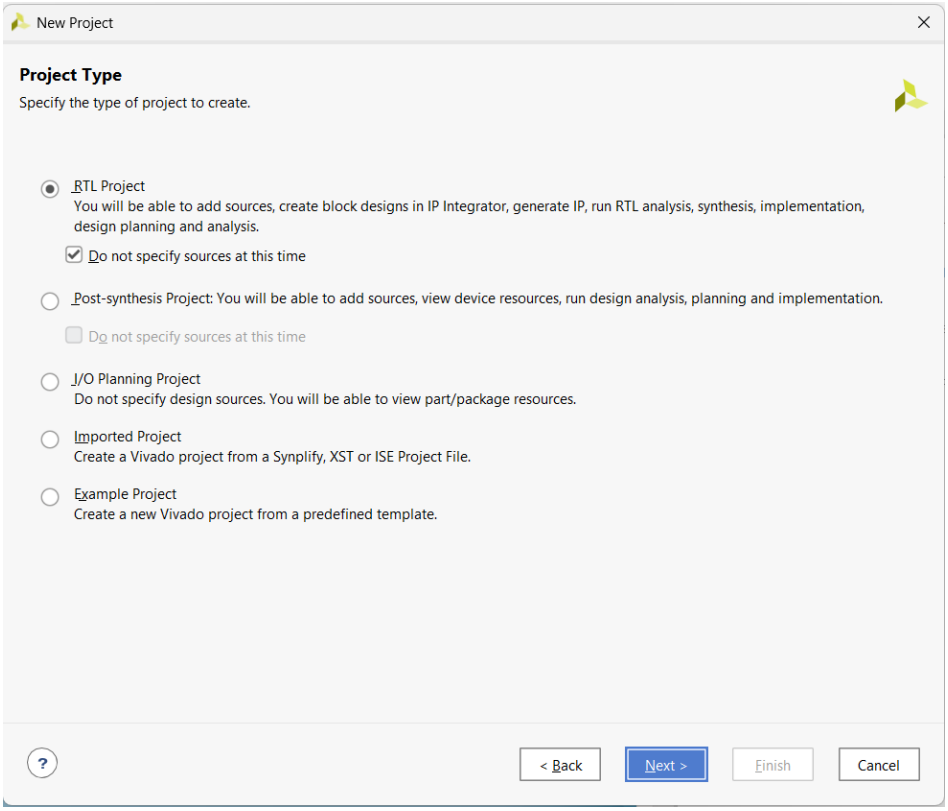
۱. نرم‌افزار Vivado را اجرا کرده و بر روی گزینه Create Project کلیک کنید.

۲. نام پروژه را LAB1_I2C قرار دهید.

۳. نوع پروژه را بر روی RTL Project تنظیم کرده و تیک گزینه Do not specify sources at this time را فعال نمایید.



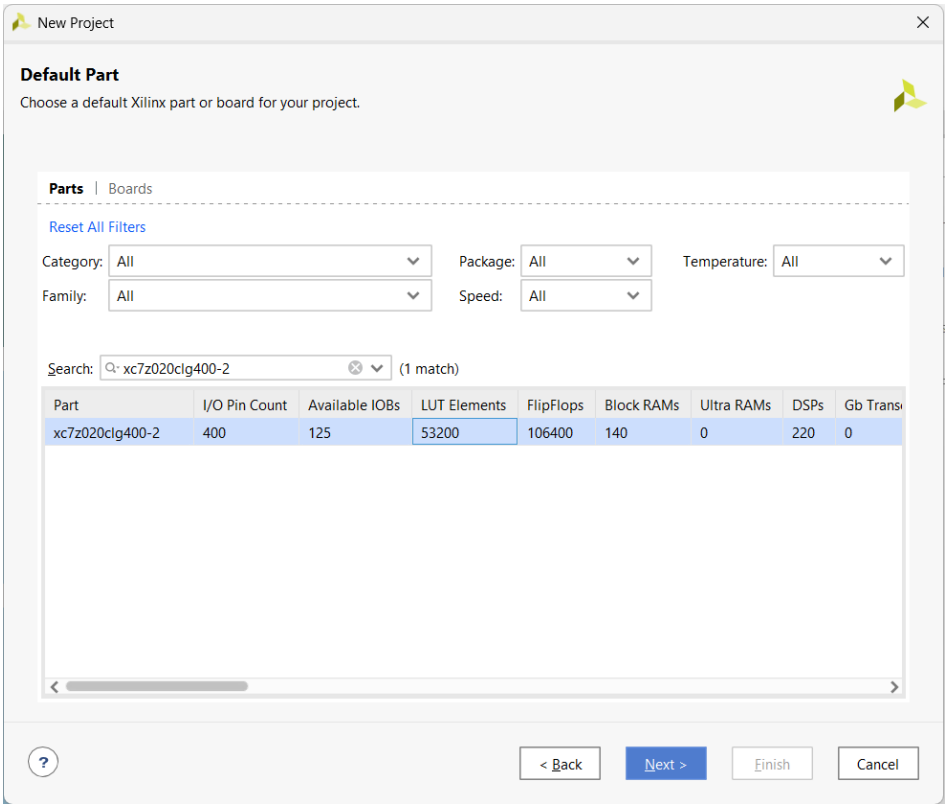
شکل ۲-۱: تعیین نام پروژه (LAB1_I2C) و مسیر ذخیره‌سازی در نرم‌افزار Vivado



شکل ۲-۲: انتخاب ساختار پروژه به صورت RTL Project

۴. در مرحله انتخاب قطعه (Default Part)، متناسب با بورد تحویل‌داده‌شده در آزمایشگاه، شماره قطعه (Part Number) مربوطه را در تب Parts جستجو و انتخاب کنید:

- **بورد** Zynq-7020: شماره قطعه xc7z020clg400-2
- **بورد** Artix-7 75T: شماره قطعه xc7a75tfgg484-2
- **بورد** Artix-7 100T: شماره قطعه xc7a100tfgg484-2



شکل ۲-۳: نمونه انتخاب شماره قطعه (Part Number) برای بورد Zynq-7020 در پنجره Default Part

۲-۲ رابط Portها و معماری ماژول i2c_master_core

ماژول i2c_master_core وظیفه تولید شرایط پروتکل و تبادل بایت‌ها روی خطوط Bus را بر عهده دارد. مشخصات کامل Portهای این هسته در جدول زیر ارائه شده است:

جدول ۲-۱: توصیف درگاه‌ها و Portهای ماژول i2c_master_core

نام Port	جهت	عرض	توصیف عملکرد
clk	ورودی	۱	سیگنال کلاک کاری سیستم (100 MHz)
rst_n	ورودی	۱	سیگنال ریست ناهمگام فعال پایین (Active-Low)
start_txn	ورودی	۱	پالس تک‌سیکلی جهت شروع ارسال یا دریافت یک بایت
gen_start	ورودی	۱	فرمان ایجاد شرط START پیش از ارسال بایت
stop_bit	ورودی	۱	فرمان ایجاد شرط STOP پس از پایان انتقال بایت
rw_bit	ورودی	۱	جهت همین بایت: ۰ یعنی Master بایت tx_data را ارسال می‌کند و ۱ یعنی Master یک بایت از Slave دریافت می‌کند. این ورودی همان بیت R/W بایت آدرس نیست؛ بیت R/W بیت صفرم tx_data است (مثلاً بایت آدرس 0x43 با rw_bit=0 ارسال می‌شود)
nack_on_read	ورودی	۱	در حالت خواندن: ۱ برای ارسال NACK توسط Master در بایت پایانی
tx_data	ورودی	۸	بایت ارسالی به قطعه Slave (بایت آدرس به همراه بیت R/W، آدرس Register یا داده)
clk_div	ورودی	۱۶	ضریب تقسیم کلاک سیستم برای تولید SCL؛ مقدار آن را بر اساس طراحی خود طوری انتخاب کنید که فرکانس SCL برابر 100 kHz شود
rx_data	خروجی	۸	بایت داده خوانده‌شده از قطعه Slave
busy	خروجی	۱	نشان‌دهنده وضعیت اشتغال هسته در حین تراکنش
ack_error	خروجی	۱	پرچم خطای عدم دریافت بیت ACK از Slave برای بایتی که Master ارسال کرده است
done	خروجی	۱	پالس تک‌سیکلی نشان‌دهنده پایان تراکنش بایت (چه با خطا و چه بدون خطا؛ وضعیت خطا از ack_error خوانده می‌شود)
scl_o	خروجی	۱	دیتای خروجی خط SCL به سمت Buffer (همواره مقدار ثابت صفر)
scl_t	خروجی	۱	سیگنال کنترل خط SCL: ۱ برای رها کردن خط (Hi-Z) و ۰ برای وصل به زمین
scl_i	ورودی	۱	مقدار منطقی خوانده‌شده از خط فیزیکی SCL
sda_o	خروجی	۱	دیتای خروجی خط SDA به سمت Buffer (همواره مقدار ثابت صفر)
sda_t	خروجی	۱	سیگنال کنترل خط SDA: ۱ برای رها کردن خط (Hi-Z) و ۰ برای وصل به زمین
sda_i	ورودی	۱	مقدار منطقی نمونه‌برداری‌شده از خط فیزیکی SDA

۲-۳ الزامات عملکردی هسته i2c_master_core

هسته i2c_master_core بایت‌محور است: هر پالس start_txn دقیقاً یک بایت را روی Bus ارسال یا دریافت می‌کند و ایجاد شرایط START و STOP با پرچم‌های همان بایت تعیین می‌شود. منطق داخلی این هسته (که خودش یک ماشین حالت است) باید بر مبنای درگاه‌های جدول ۲-۱ الزامات زیر را برآورده کند:

- در حالت بیکار (Idle)، در انتظار پالس تحریک start_txn باقی مانده و سیگنال خروجی busy را غیرفعال نگه دارد. با دریافت این پالس، سیگنال busy فعال شده و تراکنش آغاز گردد.
- در صورت فعال بودن پایه ورودی gen_start، شرایط شروع (START) پیش از ارسال داده تولید شود و در صورت فعال بودن ورودی stop_bit، پس از پایان انتقال بایت شرایط پایان (STOP) ایجاد گردد.
- تولید شکل‌موج کلاک SCL بر اساس ضریب clk_div صورت گرفته و زمان‌بندی تغییرات خط داده به گونه‌ای تنظیم شود که الزامات زمان‌های نشست (Setup) و نگه‌داری (Hold) سیگنال SDA در زمان بالابودن کلاک تامین گردد.
- تبادل داده در قالب ۸ بیت و با اولویت ارسال یا دریافت باارزش‌ترین بیت (MSB) انجام پذیرد.
- پس از انتقال ۸ بیت داده، در سیکل نهم وضعیت بیت تصدیق (ACK/NACK) مدیریت شود؛ در حالت نوشتن در صورت عدم تایید از سوی Slave پرچم خطای ack_error فعال گردد، و در حالت خواندن پاسخ مناسب متناسب با پایه nack_on_read به سمت Slave ارسال شود.
- در انتهای تراکنش، با غیرفعال شدن busy، یک پالس تک‌سیکلی روی پایه خروجی done صادر شده و ماشین به وضعیت اولیه بازگردد.

نکته طراحی: تعیین تعداد و نام‌گذاری وضعیت‌ها (States)، نحوه طراحی معماری ماشین حالت و پیاده‌سازی تقسیم کلاک کاملاً بر عهده دانشجو است؛ معیار ارزیابی، صحت زمان‌بندی سیگنال‌ها روی Bus و انطباق با پایه‌های مشخص‌شده در جدول ۲-۱ می‌باشد.

۲-۴ طراحی ماشین حالت SCCB برای خواندن شناسه‌های دوربین

هسته i2c_master_core در هر تراکنش فقط یک بایت را جابه‌جا می‌کند. برای خواندن یک Register دوربین باید چند بایت با پرچم‌های مناسب پشت سر هم روی Bus قرار گیرند. این کار بر عهده یک ماشین حالت است که شما در یک ماژول جداگانه، در لایه بالاتر از هسته طراحی می‌کنید. این ماشین حالت باید:

- با دریافت فرمان شروع، دو PID Register (0x0A) و VER (0x0B) دوربین را مطابق پروتکل SCCB (بخش ۴-۱) بخواند؛ یعنی برای هر Register یک تراکنش نوشتن دو مرحله‌ای و سپس، پس از STOP، یک تراکنش خواندن دو مرحله‌ای، بدون Repeated START انجام دهد.
- برای هر بایت، ورودی‌های هسته (tx_data، gen_start، stop_bit، rw_bit و nack_on_read) را تنظیم کرده، تراکنش آن بایت را آغاز کند و تا پایان آن صبر کند.

- مقادیر خوانده‌شده را با مقادیر مورد انتظار (0x76 و 0x73) مقایسه کرده و در صورت تطابق هر دو، یک خروجی یک بیتی را فعال کند (این خروجی در بخش دوم به LED بورد متصل می‌شود).
- رفتار مناسبی در برابر خطا (مثلاً ack_error یا مقدار نادرست شناسه‌ها) داشته باشد.
- توجه کنید که فرمان شروع در بخش دوم از یک کلید فشاری می‌آید و ممکن است برای مدت طولانی (میلیون‌ها سیکل کلاک) فعال بماند؛ با هر بار فشردن کلید، فرآیند خواندن باید تنها یک بار اجرا شود.

تعیین Portها، تعداد و نوع وضعیت‌ها و ساختار این ماشین حالت کاملاً بر عهده دانشجو است.

۵-۲ شبیه‌سازی و اعتبارسنجی

برای اعتبارسنجی طراحی، دو ماژول شبیه‌سازی در اختیار شما قرار می‌گیرد: یک ماژول Slave و یک ماژول که رفتار دوربین OV7670 را شبیه‌سازی می‌کند. با طراحی یک Testbench و قرار دادن ماژول طراحی‌شده خود در کنار ماژول ارائه‌شده، عملکرد ماژول خود را آزمایش کنید. هر دو ماژول فقط برای شبیه‌سازی هستند (قابل سنتز نیستند)، کلاک ندارند و تنها با خطوط SCL و SDA کار می‌کنند. این ماژول‌ها پروتکل و زمان‌بندی Bus را بررسی کرده و هر تخلف (مثلاً تغییر SDA در زمان بالا بودن SCL، نبود مقاومت بالاکش، درایو کردن سطح یک به جای رها کردن خط، یا START/STOP در میانه یک بایت) را با پیامی که با «ERROR» شروع می‌شود گزارش می‌کنند. شبیه‌سازی نهایی شما نباید هیچ پیام «ERROR» داشته باشد.

در هر دو مرحله، خطوط Bus را مطابق بخش ۱-۳ با tri1 تعریف کنید (در شبیه‌سازی از IOBUF استفاده نمی‌شود).

مرحله الف: آزمون هسته i2c_master_core با ماژول Slave

- ماژول Slave یک قطعه ساده I2C با آدرس ۷ بیتی 0x50 است (بایت آدرس 0xA0 برای نوشتن و 0xA1 برای خواندن). این ماژول دو Register هشت بیتی با اشاره‌گر (Pointer) 0 و 1 دارد که مقدار اولیه آن‌ها به ترتیب 0xA5 و 0x3C است. اشاره‌گر پس از هر بایت داده یک واحد افزایش می‌یابد.
- **نوشتن:** S | 0xA0 | A | PTR | A | DATA | A | P
- **خواندن:** S | 0xA0 | A | PTR | A | Sr | 0xA1 | A | DATA | NA | P
- در Testbench طراحی‌شده دست‌کم موارد زیر را آزمایش کنید: خواندن مقادیر اولیه هر دو Register، نوشتن یک مقدار دلخواه در یک Register و بازخوانی آن، و ارسال یک آدرس نادرست (در این حالت Slave پاسخ نمی‌دهد و ack_error باید فعال شود).
- دیتای دریافت‌شده روی rx_data باید دقیقاً با مقادیر Registerهای Slave منطبق باشد.

مرحله ب: آزمون ماشین حالت SCCB و هسته با ماژول شبیه‌ساز دوربین

- ماژول دوم رفتار رابط SCCB دوربین OV7670 را شبیه‌سازی می‌کند: به آدرس‌های 0x42 و 0x43 پاسخ می‌دهد، در Registerهای PID و VER مقادیر 0x76 و 0x73 را دارد، و مانند دوربین واقعی Repeated START را نمی‌پذیرد.

- ماشین حالت خود را به همراه i2c_master_core به این ماژول متصل کرده و فرآیند خواندن شناسه‌ها را شبیه‌سازی کنید. خروجی تطابق شناسه‌ها باید فعال شود.
- مقادیر شناسه در این ماژول با پارامترهای PID_VALUE و VER_VALUE قابل تغییر است. با یک مقدار نادرست نیز شبیه‌سازی کنید و نشان دهید که ماشین حالت شما در این حالت موفقیت را اعلام نمی‌کند.

با اجرای دستور Run Behavioral Simulation در نرم‌افزار Vivado، شکل‌موج سیگنال‌ها، زمان‌بندی آن‌ها، صحت ارسال آدرس، دریافت بایت‌ها و بیت‌های ACK/NACK را در هر دو مرحله بررسی کرده و برای گزارش کار خود ذخیره نمایید.

۳ طراحی سخت‌افزار در Vivado، پیاده‌سازی روی بورد و راه‌اندازی دوربین (بخش دوم)

پس از اطمینان از صحت عملکرد هسته I2C Master در محیط شبیه‌سازی، وارد مرحله پیاده‌سازی عملی مدار بر روی تراشه FPGA و تست واقعی با دوربین می‌شویم. هدف در این بخش این است که بتوانیم با فشردن یک کلید روی بورد، شناسه‌های سنسور دوربین (PID و VER) را از طریق پروتکل سریال بخوانیم و در صورت تطابق، روشن شدن یک نشانگر LED را به عنوان تایید موفقیت مشاهده کنیم.

برای تحقق این هدف روی سخت‌افزار واقعی، هسته ارتباطی I2C به تنهایی کافی نیست و نیازمند یک سامانه یکپارچه هستیم:

- سنسور دوربین برای راه‌اندازی مدارهای داخلی و مبدل‌های تصویر، به کلاک ورودی با فرکانسی بین ۱۰ تا ۴۸ مگاهرتز (XCLK) نیاز دارد. اسیلاتور بورد Zynq-7020 فرکانسی برابر با ۵۰ مگاهرتز و خروجی تک‌سَر (single-ended) دارد؛ در بوردهای Artix-7 نیز از اسیلاتور ۲۰۰ مگاهرتزی با خروجی تفاضلی (differential) استفاده شده است. از آنجا که هسته کنترلی با کلاک ۱۰۰ مگاهرتز کار می‌کند، برای تولید کلاک‌های موردنیاز از ماژول مدیریت کلاک (Clocking Wizard) استفاده می‌کنیم.
- فرمان آغاز خواندن با یک کلید فشاری مکانیکی اعمال می‌شود. برای حذف جهش‌ها و نوسانات مکانیکی کلید، به یک مدار حذف لرزش (button_debouncer) و گیت‌های منطقی کنترلی نیاز داریم. مدار حذف لرزش ورودی کلید را با کلاک سیستم همگام کرده و تنها زمانی خروجی خود را تغییر می‌دهد که ورودی برای مدت مشخصی (چند میلی‌ثانیه) ثابت مانده باشد.
- با توجه به ماهیت دوطرفه و درین‌باز خطوط ارتباطی، برای اتصال سیگنال‌های داخلی به پایه‌های فیزیکی بورد، از بافرهای دوطرفه IOBUF (در قالب ماژول my_IOBUF، بخش ۳-۱) استفاده می‌کنیم.
- ماشین حالت SCCB که در بخش ۴-۲ طراحی کرده‌اید، با فشردن کلید فرایند خواندن شناسه‌ها را از طریق هسته I2C اجرا می‌کند و در صورت تأیید شناسه‌ها، خروجی LED را فعال می‌سازد.

محیط Block Design در Vivado IP Integrator به ما این امکان را می‌دهد که تمام این ماژول‌های مستقل را در قالب یک سامانه سخت‌افزاری ماژولار و یکپارچه در کنار یکدیگر قرار داده، اتصالات میان آن‌ها را برقرار کرده و پس از اعمال فایل Constraints و تولید Bitstream، مدار را بر روی بورد Program و تست نماییم.

۳-۱ ساخت Block Design و افزودن ماژول‌ها

۱. از پنل Flow Navigator گزینه Create Block Design را انتخاب کرده و نام آن را OV7670_config_design تعیین کنید.

۲. ماژول‌های Verilog زیر باید به عنوان Design Source در پروژه شما موجود باشند:

- هسته i2c_master_core و ماشین حالت SCCB که در بخش اول طراحی و شبیه‌سازی کرده‌اید.
- ماژول طراحی‌شده حذف لرزش کلید (button_debouncer).
- ماژول طراحی‌شده my_IOBUF که تنها یک نمونه از IOBUF را در بر دارد (بخش ۳-۱).

ماژول‌های شبیه‌سازی (ماژول Slave و ماژول شبیه‌ساز دوربین) و Testbenchها نباید در Design Sources باشند؛ آن‌ها را فقط در Simulation Sources نگه دارید.

۳. با راست‌کلیک در صفحه دیاگرام و انتخاب گزینه Add Module، ماژول‌ها را اضافه کنید. از ماژول my_IOBUF باید دو نمونه با نام‌های iobuf_scl و iobuf_sda ایجاد شود.

۳-۲ ساخت Portهای ورودی و خروجی

پس از افزودن ماژول‌های موردنیاز به محیط Block Design، باید Portهای خارجی تعریف شوند تا سیگنال‌های داخلی به پایه‌های فیزیکی بورد متصل گردند. برای این منظور، در محیط Block Design می‌توان با راست‌کلیک بر روی پایه سیگنال موردنظر و انتخاب گزینه Make External، Port متناظر را ایجاد کرد. همچنین می‌توان با راست کلیک و اعمال گزینه ...Create Port نیز Portهای ورودی و خروجی تعریف کرد. Portهای موردنیاز به صورت زیر تعریف می‌شوند:

۱. **Portهای ورودی:**

- reset: ورودی ریست سیستم که به کلید ریست بورد (فعال پایین، جدول ۳-۱) متصل می‌شود. این ورودی به ریست Clocking Wizard و به ورودی rst_n سایر ماژول‌ها وصل می‌گردد.
- pushButton_1: ورودی کلید فشاری برای آغاز فرآیند خواندن شناسه دوربین.
- clk_in: کلاک ورودی اصلی سیستم در بوردهایی با کلاک تک‌سَر (Single-Ended).
- در صورت استفاده از بوردهای Artix-7 با کلاک تفاضلی (Differential)، دو Port ورودی با نام‌های clk_in_P و clk_in_N ایجاد شوند.

۲. **Portهای دوطرفه** (Inout):

- scl: خط کلاک ارتباط سریال I2C.
- sda: خط داده دوطرفه ارتباط سریال I2C.

۳. **Portهای خروجی:**

- XCLK_out: خروجی کلاک موردنیاز برای راه‌اندازی سنسور دوربین OV7670.
- LED_out: خروجی نشانگر LED جهت نمایش موفقیت‌آمیز بودن فرآیند خواندن شناسه دوربین.

در بورد Zynq-7020 که از کلاک ورودی تک‌سَر استفاده می‌کند، تنها clk_in به عنوان ورودی کلاک ایجاد می‌شود؛ در حالی که در بوردهای Artix-7 با کلاک تفاضلی، هر دو ورودی clk_in_P و clk_in_N باید به عنوان یک زوج تفاضلی تعریف شوند. خطوط scl و sda نیز به دلیل ماهیت دوطرفه و ساختار Open-Drain به صورت Inout تعریف می‌شوند و از طریق ماژول‌های IOBUF به هسته I2C متصل خواهند شد.

پس از ایجاد این Portها، اتصال کلاک ورودی به ماژول Clocking Wizard و اتصال سایر سیگنال‌ها به ماژول‌های مربوطه انجام می‌شود.

۳-۳ پیکربندی ماژول Clocking Wizard

سنسور OV7670 برای عملکرد صحیح مبدل‌های داخلی نیازمند فرکانس کلاک مشخصی است:

۱. Clocking Wizard Block را اضافه کرده و روی آن دابل‌کلیک کنید.

۲. در تب Clocking Options، فرکانس کلاک ورودی اصلی (Primary) را مطابق اسیلاتور بورد تنظیم کنید.

۳. در تب Output Clocks، دو کلاک خروجی را فعال نمایید:

- کلاک خروجی اول (clk_100MHz): با فرکانس 100.00 MHz به عنوان کلاک کاری هسته و ماشین حالت.
- کلاک خروجی دوم (clk_24MHz): با فرکانس نامی 24.00 MHz جهت اتصال به XCLK Port دوربین.

۴. نوع پایه ریست را روی Active Low قرار دهید.

۴-۳ مدار راه‌انداز کلید و کنترل ریست

جهت اعمال فرمان شروع با فشردن کلید فشاری:

۱. دو Block منطقی Utility Vector Logic اضافه کنید:

- Block اول: با عملگر not و عرض ۱ بیت (برای معکوس‌سازی پلاریته کلید بورد).
- Block دوم: با عملگر and و عرض ۱ بیت.

۲. Port ورودی pushButton_1 به گیت NOT و خروجی آن به ورودی ماژول button_debouncer متصل می‌گردد تا جهش‌های مکانیکی فیلتر شوند.

۳. خروجی ماژول Debouncer با سیگنال locked ماژول کلاک توسط گیت AND ضرب منطقی می‌شود تا از پایداری کامل کلاک پیش از آغاز تراکنش اطمینان حاصل گردد.

۴. خروجی گیت AND به ورودی فرمان شروع ماشین حالت SCCB شما متصل می‌شود.

۳-۵ اتصال Bufferهای دوطرفه IOBUF و Portهای خارجی

۱. **خط کلاک سریال** (SCL):

- scl_o Port هسته به ورودی io_in ماژول iobuf_scl وصل می‌شود.
- scl_t Port هسته به ورودی io_t ماژول iobuf_scl متصل می‌گردد.
- خروجی io_out ماژول Buffer به ورودی scl_i هسته فیدبک می‌شود.
- پایه io_io با کلیک‌راست و گزینه Make External به Port خارجی با نام scl تبدیل می‌گردد.

۲. **خط دیتای سریال** (SDA):

- sda_o Port هسته به ورودی io_in ماژول iobuf_sda متصل می‌شود.
- sda_t Port هسته به ورودی io_t ماژول iobuf_sda وصل می‌گردد.
- خروجی io_out ماژول Buffer به ورودی sda_i هسته فیدبک می‌شود.
- پایه io_io به Port خارجی با نام sda تبدیل می‌گردد.

۳. خروجی ماشین حالت که نشان‌دهنده صحت داده‌های دریافتی است به خروجی مربوط به LED متصل می‌شود.

۴. خروجی ۲۴ مگاهرتز ماژول کلاک (clk_24MHz) با نام XCLK_out به صورت Port خارجی درمی‌آید تا به کلاک ورودی دوربین متصل شود.

۳-۶ افزودن و پیکربندی ماژول ILA (بخش امتیازی – Bonus)

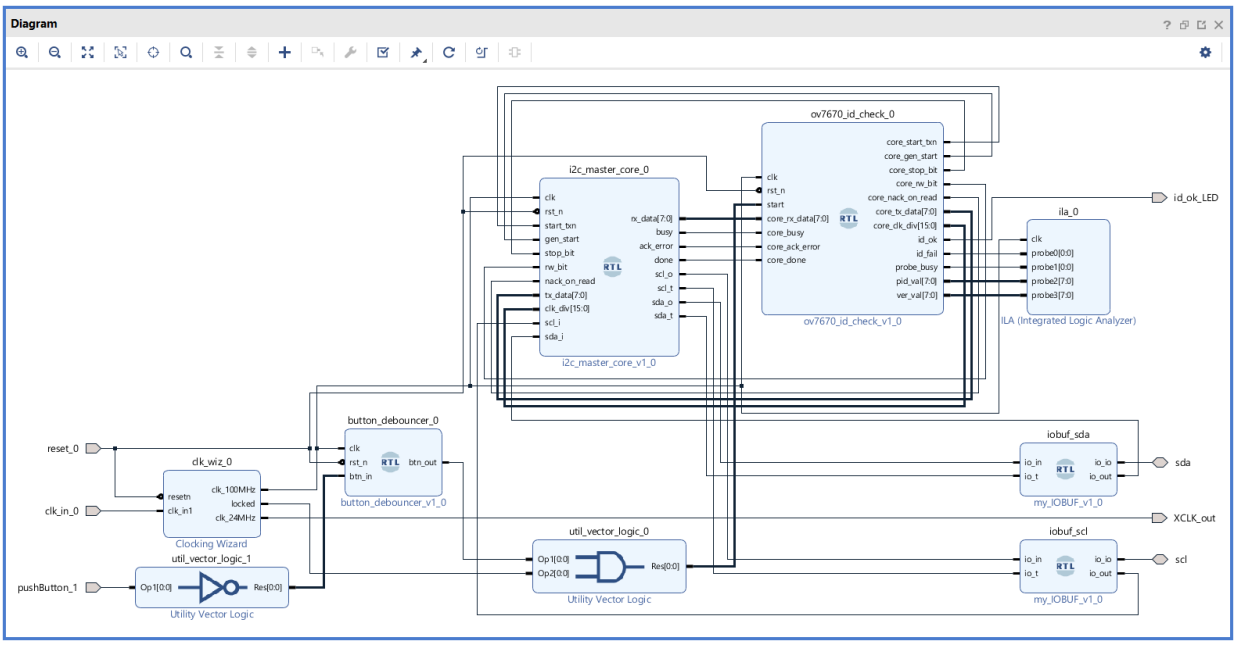
جهت مانیتورینگ سیگنال‌ها روی بورد، می‌توانید یک ماژول ILA به Block Design اضافه کنید:

- کلاک کاری ماژول: متصل به clk_100MHz.
- تعداد و عرض Probeها را بر اساس سیگنال‌هایی که می‌خواهید مشاهده کنید تعیین کنید؛ دست‌کم مقادیر خوانده‌شده از Registerهای PID و VER و وضعیت موفقیت یا خطای ماشین حالت باید قابل مشاهده باشند.

۳-۷ اعتبارسنجی طرح و ایجاد فایل پوششی (HDL Wrapper)

۱. با فشردن کلید F6 (Validate Design) اتصالات دیاگرام را ارزیابی کرده و خطاهای احتمالی را رفع نمایید.

۲. در تب Sources بر روی فایل OV7670_config_design.bd کلیک‌راست کرده و گزینه Create HDL Wrapper را اجرا کنید.



شکل ۳-۱: نمونه‌ای از Block Diagram سامانه سخت‌افزاری در محیط Vivado IP Integrator (نام و Portهای ماشین حالت در طراحی شما می‌تواند متفاوت باشد)

۳-۸ افزودن فایل Constraints و تولید Bitstream

برای تعیین اینکه کدام پایه‌های FPGA به چه Portهایی از سیستم طراحی شده در محیط Block Design متصل شوند، نیازمند خواندن فایل شماتیک بورد خود هستید. در فایل‌های شماتیک توضیحات مهمی مانند پایه کلاک ورودی، ریست ورودی، پایه‌های متصل به کانکتورهای برد، ولتاژ هر بانک از FPGA و ... قرار دارد.

در بخش RTL ANALYSIS با انتخاب گزینه Open Elaborated Design و باز کردن پنجره I/O Ports، پنجره‌ای جهت انتخاب پایه‌های فیزیکی FPGA و تعیین استاندارد ولتاژ، تنظیمات مقاومت بالاکش یا پایین‌کش (Pull-type) باز می‌شود. مطابق شماتیک داده شده برای هر بورد، پایه‌ها باید انتخاب شوند. پس از انتخاب و ذخیره، فایل xdc تولید می‌شود.

جدول ۳-۱: جدول نمونه تخصیص پایه‌ها روی تراشه Zynq-7020 (xc7z020clg400-2)

نام Port	شماره Pin	استاندارد	توضیحات اتصال
clk_in_0	U18	LVCMOS33	کلاک ورودی ۵۰ مگاهرتز اسیلاتور بورد
reset_0	U19	LVCMOS33	کلید ریست فعال پایین (Active-Low)
pushButton_1	G14	LVCMOS33	کلید فرمان آغاز فرآیند خواندن شناسه دوربین
id_ok_LED	R19	LVCMOS33	نشانگر تایید موفقیت و تطابق شناسه‌های دوربین
XCLK_out	H16	LVCMOS33	خروجی کلاک ۲۴ مگاهرتز اعمالی به سنسور دوربین
scl	E17	LVCMOS33	خط کلاک سریال با مقاومت بالاکش داخلی فعال
sda	D18	LVCMOS33	خط دیتای سریال با مقاومت بالاکش داخلی فعال

پس از کنترل صحت اتصالات، بر روی گزینه Generate Bitstream در پنل سمت چپ کلیک نمایید تا کلیه مراحل سنتز[1]، جایابی[2] و سیم‌کشی[3] سخت‌افزار به صورت خودکار اجرا شده و فایل پیکربندی (bit.) تولید گردد.

۳-۹ Program کردن بورد و تست عملی با دوربین

۱. ماژول دوربین OV7670 را با دقت به پایه‌های هدر مربوطه روی بورد متصل کنید.

۲. بورد را روشن کرده و از طریق منوی Open Hardware Manager -> Auto Connect ارتباط JTAG را برقرار نمایید.

۳. با اجرای گزینه Program Device فایل Bitstream را روی تراشه بارگذاری کنید.

۴. **تست اصلی با نشانگر LED:** با فشردن کلید pushButton_1، ماشین حالت مراحل خواندن داده از طریق پروتکل SCCB را آغاز می‌کند. در صورت خواندن صحیح مقادیر 0x76 و 0x73، نشانگر LED روشن می‌گردد که بیانگر صحت کامل عملکرد سخت‌افزار و اتمام موفقیت‌آمیز تراکنش (Done) است.

۵. **تست تکمیلی با ماژول ILA (بخش امتیازی):** در صورت پیاده‌سازی ILA، یک شرط تریگر مناسب انتخاب کنید (انتخاب آن بر عهده شماست) و با فشردن کلید، مقادیر خوانده‌شده از Registerهای PID و VER را در Vivado Hardware Manager مشاهده و ثبت کنید.

۴ موارد تحویلی گزارش کار و ارزیابی عملکرد (Deliverables)

هدف از تدوین گزارش کار، مستندسازی دقیق فرآیند مهندسی، تحلیل نتایج به‌دست‌آمده از شبیه‌سازی و ارزیابی عملکرد مدار روی بورد سخت‌افزاری است. علاوه بر تحویل گزارش مکتوب، تسلط بر ساختار کلی سیستم و پاسخ‌گویی به سوالات شفاهی در رابطه با نقش هر یک از اجزای سخت‌افزاری و شکل‌موج سیگنال‌ها، ملاک اصلی ارزیابی در جلسه حضوری آزمایشگاه خواهد بود.

دانشجویان محترم موظف هستند در گزارش کار ارسالی موارد زیر را مستند نمایند:

۱. **تسلط و پاسخ‌گویی به سوالات شفاهی Block Diagram سخت‌افزاری:** در جلسه ارزیابی حضوری، از تک‌تک اجزای Block Diagram سخت‌افزاری پیاده‌سازی‌شده (نحوه زمان‌بندی هسته I2C، عملکرد ماشین حالت، نقش Bufferهای دوطرفه IOBUF، ضرورت وجود Clocking Wizard، ساختار مقاومت‌های Pull-up و ماژول Debouncer) سوال پرسیده خواهد شد.

۲. **کدهای منبع:** تمام فایل‌های Verilog طراحی‌شده (i2c_master_core، ماشین حالت button_debouncer، SCCB، my_IOBUF)، Testbenchهای نوشته‌شده و فایل Constraints.

۳. **تحلیل و ثبت شکل‌موج شبیه‌سازی:** ارائه شکل‌موج‌های شبیه‌سازی هر دو مرحله. در مرحله الف (هسته با ماژول Slave): صحت تولید کلاک SCL با فرکانس 100 kHz و زمان‌بندی سیگنال‌ها، صدور

[1] Synthesis
[2] Placement
[3] Routing

شرایط START/STOP و Repeated START، ارسال آدرس، نوشتن و بازخوانی Registerهای Slave روی rx_data، تبادل بیت‌های ACK/NACK و فعال شدن ack_error با آدرس نادرست. در مرحله ب (ماشین حالت با ماژول شبیه‌ساز دوربین): دنباله کامل خواندن PID و VER، فعال شدن خروجی تطابق با شناسه‌های درست و فعال نشدن آن با شناسه نادرست. گزارش شبیه‌سازی بدون پیام «ERROR» را نیز ضمیمه کنید.

۴. **اسکرین‌شات از Block Diagram نهایی:** اسکرین‌شات کامل و خوانا از محیط IP Integrator شامل تمامی Blockها و سیم‌بندی‌ها.

۵. **مستندسازی تخصیص پایه‌ها و مقاومت‌های بالاکش:** ارائه تصویر از جدول I/O Ports یا متن فایل Constraints با هایلایت فعال بودن ویژگی PULLUP = true روی پایه‌های scl و sda.

۶. **مستندسازی تست عملی سخت‌افزار:** ارائه تصویر از بورد آزمایشگاهی در حالت روشن بودن نشانگر LED پس از تحریک کلید، و در صورت انجام بخش امتیازی، اسکرین‌شات از پنجره ILA با مقادیر بازخوانی‌شده 0x76 و 0x73.

۷. **پاسخ به سوالات تحلیلی و تئوری:**

- چرا در پروتکل‌های I2C و SCCB از ساختار کلکتور/درین باز (Open-Drain) استفاده می‌شود؟ در صورت عدم فعال‌سازی مقاومت بالاکش داخلی (Internal Pull-up) در تراشه FPGA، وضعیت ولتاژ خطوط چگونه خواهد بود و چه اثری بر ارتباط با سنسور می‌گذارد؟
- چرا فرکانس کلاک XCLK اعمالی به دوربین OV7670 در یک بازه فرکانسی خاص تنظیم می‌شود؟ اعمال فرکانس‌های نامتعارف چه اثری بر کارکرد پردازش سیگنال داخلی سنسور دارد؟
- نحوه تامین زمان‌های نشست (Setup) و نگه‌داری (Hold) داده در زمان‌بندی کلاک SCL و طراحی ماشین حالت را تشریح کنید.
- تفاوت فرآیند خواندن Register در پروتکل SCCB نسبت به پروتکل استاندارد I2C (به ویژه در رابطه با عدم پشتیبانی از شرط Repeated START) را توضیح دهید.

★ یادداشت‌های تکمیلی و راهنمای سخت‌افزاری (Hardware Notes)

- **اتصال ماژول دوربین:** هنگام اتصال دوربین OV7670 به کانکتور مادگی روی بورد، به شماره و نام Pinها (به ویژه Pinهای تغذیه 3.3V و زمین GND) دقت مضاعف فرمایید؛ اتصال وارونه ماژول ممکن است منجر به آسیب دائمی سنسور شود.
- **پایه‌های کنترلی دوربین:** پیش از تست، بررسی کنید که XCLK به دوربین می‌رسد، #RESET در سطح یک و PWDN در سطح صفر است (بخش ۴-۱).